

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

**Redovne studije
Smjer : “Inženjering Informacionih Tehnologija“**

**Predmet:
Softverski inženjering**

PROJEKTOVANJE SISTEMA

(Seminarski rad)

**Predmetni nastavnik
Doc. dr Dalibor Drljača**

**Student: Ivan Pavlović
Br. Indeksa: 92-20 RITP-S**

Banja Luka, jun 2026

SADRŽAJ

UVOD	1
1. ISPORUKA I ODRŽAVANJE SOFTVERA	2
1.1. Životni ciklus softvera	2
1.2. Proces isporuke softvera	2
1.3. Kvalitet softvera i održavanje	3
2. PROCES ISPORUKE SOFTVERA	4
2.1. Planiranje izdanja i upravljanje promjenama.....	4
2.2. Kontinuirana integracija i kontinuirana isporuka (CI/CD)	4
2.3. Raspoređivanje (deploy) i strategije puštanja u rad.....	6
3. ODRŽAVANJE I EVOLUCIJA SOFTVERA	7
3.1. Vrste održavanja i tipični okidači promjena	7
3.2. Upravljanje greškama, incidentima i povratnim informacijama korisnika.....	8
3.3. Tehnički dug, refaktorisanje i modernizacija sistema.....	9
4. ORGANIZACIJA PODRŠKE I MJERENJE USPJEŠNOSTI.....	10
4.1. Operativna podrška i prakse iz SRE i ITIL okvira	10
4.2. Praćenje, logovanje i metrika isporuke i stabilnosti	11
4.3. Bezbijednost i usklađenost kao dio isporuke i održavanja	12
ZAKLJUČAK.....	14
POPIS SLIKA	15
CITATNI IZVORI	16

UVOD

Isporuka i održavanje softvera su teme koje na prvi pogled djeluju “operativno”, kao nešto što dolazi tek poslije programiranja. Međutim, u praksi se najveći dio životnog vijeka softvera provede upravo nakon prve verzije, kroz korištenje, popravke, dorade i prilagođavanja. Zbog toga se isporuka i održavanje ne mogu posmatrati kao sporedne aktivnosti, nego kao važan dio softverskog inženjeringa.

Kad se kaže isporuka, ljudi često pomisle samo na trenutak kad se aplikacija “postavi na server” ili kad korisnik preuzme novu verziju. U stvarnosti, isporuka uključuje mnogo više: pripremu izdanja, provjere kvaliteta, upravljanje promjenama, praćenje rizika, plan povratka na staru verziju, kao i komunikaciju sa korisnicima. Ako se to radi neorganizovano, posljedica su česta “gašenja” sistema, gubitak povjerenja i skupi hitni popravci.

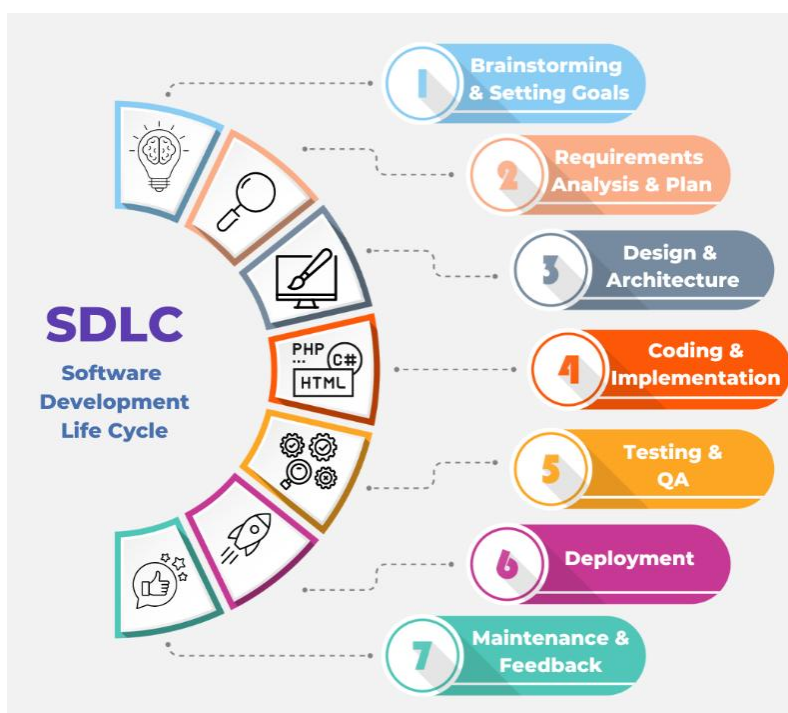
Održavanje je, s druge strane, širi pojam od prostog ispravljanja grešaka. Softver se mijenja jer se mijenjaju i potrebe korisnika, zakon, tržište, uređaji, bezbjednosne prijetnje i očekivanja o performansama. Standardi životnog ciklusa i održavanja naglašavaju da je održavanje proces sa svojim aktivnostima, odgovornostima i rezultatima, a ne samo “usluga kad nešto pukne.

1. ISPORUKA I ODRŽAVANJE SOFTVERA

1.1. Životni ciklus softvera

Softver prolazi kroz više faza tokom svog životnog vijeka. Proces počinje analizom zahtjeva i planiranjem, nakon čega slijede projektovanje, implementacija, testiranje, isporuka i korištenje. Kada se softver počne koristiti, javlja se potreba za održavanjem, ispravljanjem grešaka i dodavanjem novih funkcionalnosti.

Prelazak iz razvojnog okruženja u produkciju predstavlja važan korak u životnom ciklusu softvera. Tada se aplikacija prvi put koristi sa stvarnim podacima i od strane stvarnih korisnika. Zbog toga je potrebno pažljivo pripremiti svaku novu verziju kako bi se smanjio rizik od grešaka i prekida rada sistema.



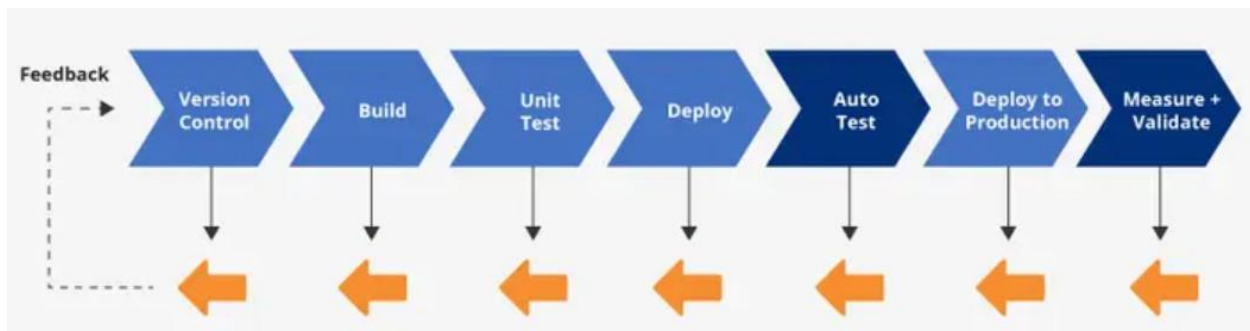
Slika 1 - 7 faza životnog ciklusa razvoja softvera (SDLC)

Bez obzira na to da li se koristi tradicionalni ili agilni pristup razvoju, neophodno je kontrolisati promjene i provjeravati kvalitet softvera. Tim mora znati koje su izmjene napravljene, ko ih je odobrio i da li su pravilno testirane prije puštanja u rad. Savremeni razvoj softvera zasniva se na ponavljanju istih aktivnosti kroz više iteracija. Nakon svake isporuke prikupljaju se povratne informacije korisnika, vrše potrebne izmjene i priprema nova verzija sistema.

1.2. Proces isporuke softvera

Isporuka softvera podrazumijeva pripremu i puštanje određene verzije aplikacije u radno okruženje. Ovaj proces obuhvata instalaciju, konfiguraciju, pripremu potrebnih datoteka i provjeru da sistem može pravilno funkcionisati kod korisnika.

Problemi tokom isporuke često nisu povezani sa samim programskim kodom. Greške mogu nastati zbog neodgovarajućih konfiguracija, različitih verzija biblioteka ili pogrešno podešenog okruženja. Zbog toga je važno da postupak isporuke bude jasno definisan i kontrolisan. U praksi se koristi upravljanje verzijama i konfiguracijom kako bi se pratilo koje promjene su uvedene u sistem. Na taj način moguće je utvrditi koja se verzija koristi, koje funkcionalnosti sadrži i kada je postavljena u produkciju.



Slika 2 - Dijagram DevOps procesa isporuke softvera

Prije puštanja nove verzije obično se provode dodatne provjere kako bi se potvrdilo da sistem radi očekivano. Takođe se priprema plan za slučaj da dođe do problema nakon isporuke, kako bi se sistem mogao brzo vratiti u prethodno stabilno stanje.

1.3. Kvalitet softvera i održavanje

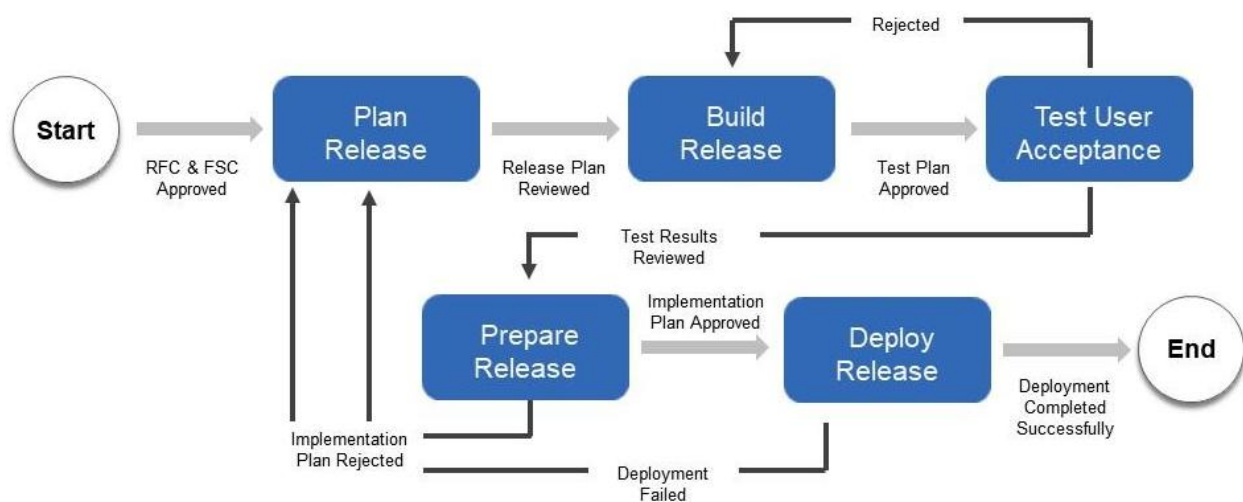
Kvalitet softvera zavisi od načina na koji je sistem projektovan, implementiran i testiran. Važne osobine kvalitetnog softvera su pouzdanost, bezbjednost, dobre performanse i jednostavno održavanje. Održavanje je lakše kada je kod pregledan, dobro organizovan i dokumentovan. U suprotnom, čak i male izmjene mogu zahtijevati mnogo vremena jer je teško procijeniti njihov uticaj na ostatak sistema. Kvalitet ima veliki značaj i tokom isporuke. Aplikacija koja se teško instalira ili zahtijeva složena podešavanja može stvarati probleme korisnicima i administratorima sistema. Zbog toga je važno da postupak instalacije i konfiguracije bude što jednostavniji i pouzdaniji.

Najveći dio životnog vijeka softvera provodi se u upotrebi. Zbog toga je potrebno pratiti rad sistema, evidentirati greške i reagovati na incidente kada se pojave. Redovno održavanje omogućava da softver ostane funkcionalan i prilagođen novim zahtjevima korisnika. Prije objavljivanja nove verzije obično se definišu minimalni uslovi koje sistem mora ispuniti. Na taj način se postiže ravnoteža između brzine razvoja i stabilnog rada aplikacije.

2. PROCES ISPORUKE SOFTVERA

2.1. Planiranje izdanja i upravljanje promjenama

Planiranje izdanja podrazumijeva da se unaprijed zna šta ulazi u verziju, koji su prioriteti, koji su rizici i kako se verzija uvodi. U praksi, čak i mali tim treba neku vrstu plana izdanja, makar jednostavan dokument ili zapis u alatu (npr. backlog i release notes). Poenta nije birokratija, nego jasnoća: da svi u timu imaju istu sliku o tome šta se isporučuje.



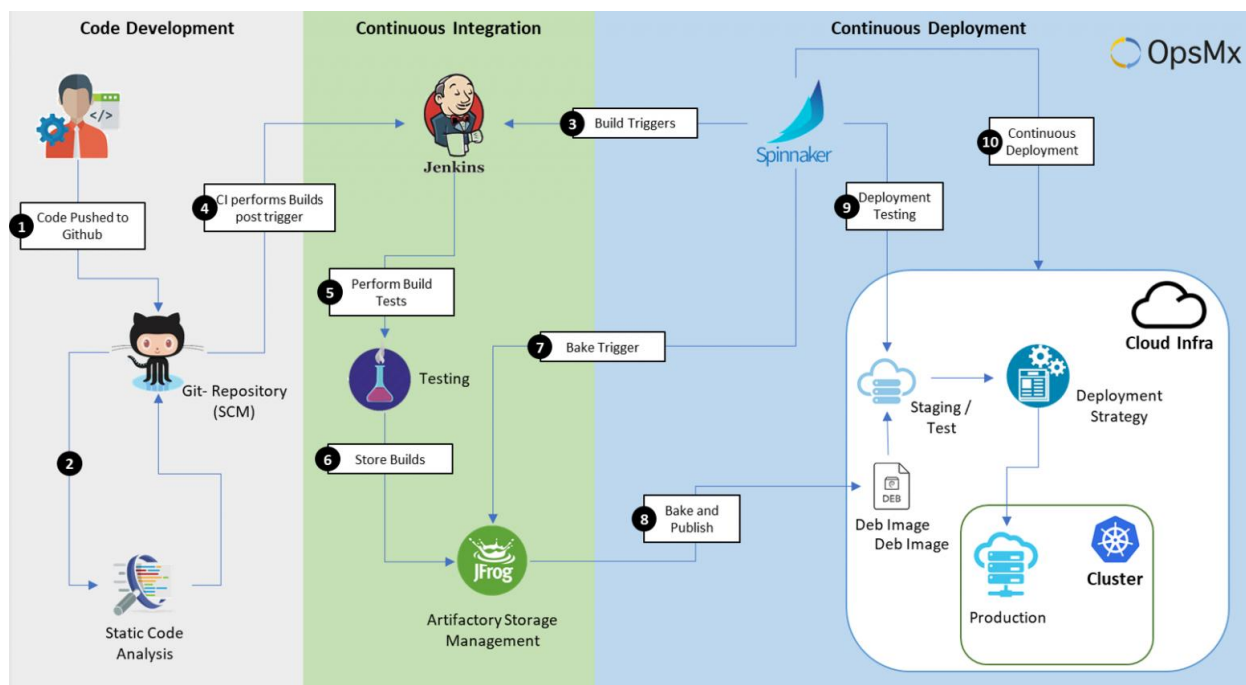
Slika 3 - Upravljanje izdanjima sa dijagramom toka u pet koraka

Upravljanje promjenama je usko povezano s tim. Promjena može biti nova funkcionalnost, ispravka greške, poboljšanje performansi, bezbjednosni patch ili promjena konfiguracije. Problem nastaje kad promjene ulaze “usput”, bez evidencije i bez provjere uticaja. U konfiguracionom upravljanju se naglašava da promjene moraju imati kontrolu, status i provjeru, kako bi se izbjeglo da sistem postane skup nepovezanih izmjena. To se posebno vidi kada više ljudi radi na istom proizvodu. Dobar dio planiranja izdanja je i procjena rizika. „U knjigama o softverskom inženjeringu se ističe da se rizici moraju prepoznati i pratiti, jer se projektni problemi često ne dešavaju zbog jedne velike greške, nego zbog niza malih rizika koji se zanemare” [1]. Na primjer, “sitna” promjena u biblioteci može izazvati nekompatibilnost, ili “sitna” promjena u bazi može učiniti stare podatke neupotrebljivim. Još jedna bitna stvar je komunikacija. Isporuca koja nije ispraćena dobrim obavještenjem može izazvati konfuziju i dodatne tikete podrške. Korisnici treba da znaju šta je novo, šta se promijenilo, postoji li promjena u ponašanju sistema i šta treba uraditi sa njihove strane. U ozbiljnijim sistemima, release notes i evidencija promjena su standardni dio procesa i često služe i kao dokaz u slučaju audita.

2.2. Kontinuirana integracija i kontinuirana isporuka (CI/CD)

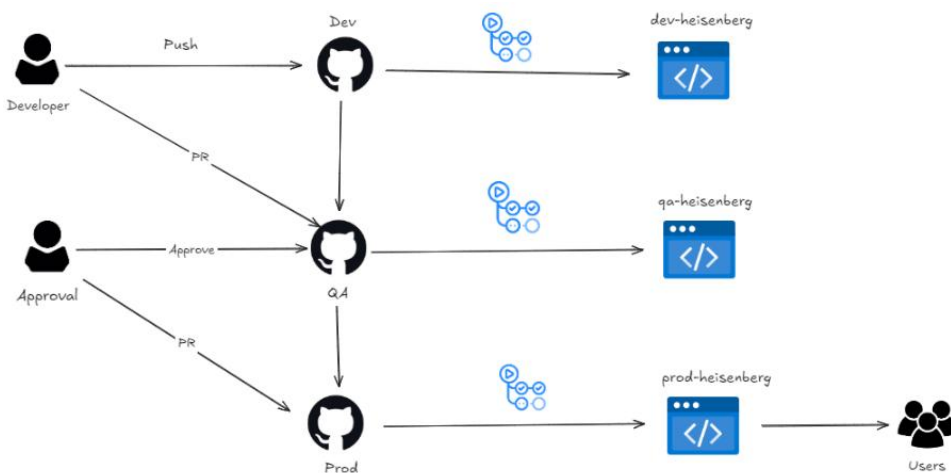
Kontinuirana integracija (CI) je praksa u kojoj se promjene koda često spajaju u zajedničku bazu, uz automatizovane provjere kao što su build i testovi. Kontinuirana isporuka (CD) ide korak dalje i pokušava da proces od promjene koda do spremnosti za isporuku bude automatizovan i ponovljiv.

„U knjizi o kontinuiranoj isporuci se naglašava da automatizacija build-a, testiranja i raspoređivanja smanjuje rizik i omogućava češća, manja izdanja, što obično znači lakšu kontrolu i brži povrat informacija” [2]. Kad tim ima automatizovane provjere, onda se greške hvataju ranije. Na primjer, ako se promijeni dio koda koji utiče na prijavu korisnika, testovi mogu odmah pokazati da je nešto polomljeno. To je bolje nego da se greška otkrije tek kad korisnici prijave problem.



Slika 4 - Dijagram CI/CD procesa isporuke softvera

CI/CD je važan i za održavanje. Ako se sistem može brzo i sigurno isporučiti, onda se i popravke mogu brže isporučiti. U praksi, brzina isporuke je bitna kada se pojavi bezbjednosna ranjivost ili kritična greška. U takvim situacijama tim često nema luksuz da “čekamo sljedeći mjesec”, nego mora brzo napraviti patch, testirati i isporučiti. Dobar CI/CD proces tada nije samo pogodnost, nego zaštita.



Slika 5 - CI/CD proces sa GitHub Actions na Azure Web App

Naravno, automatizacija ne znači da se sve radi bez razmišljanja. „Automatizovani pipeline mora biti dobro dizajniran: mora imati jasne korake, jasne kriterijume prolaza i mora se održavati kao i ostatak sistema. Ako pipeline postane spor, nepouzdan ili pun izuzetaka, ljudi će ga zaobilaziti, i onda se vraćamo na ručni, rizični način isporuke. U literaturi se često naglašava da procesi i alati moraju podržati tim, a ne postati prepreka” [1].

2.3. Raspoređivanje (deploy) i strategije puštanja u rad

Raspoređivanje ili deploy je faza u kojoj se softver fizički ili logički uvodi u ciljno okruženje: server, cloud, mobilni uređaj, terminal, ili neki drugi sistem. U praksi, deploy nije samo “kopiranje fajlova”, nego koordinacija koda, konfiguracije, baze podataka, zavisnosti i infrastrukture. Čak i kad je aplikacija mala, postoje pitanja: gdje se čuva konfiguracija, kako se upravlja tajnama (lozinke, tokeni), kako se migrira baza, kako se vraća stara verzija ako treba.

“Jedna važna ideja je da se promjene uvode postepeno kad god je to moguće. Na primjer, tim može prvo pustiti novu verziju manjom grupom korisnika, ili na jednom serveru, pa tek onda proširiti. Iako ovakve strategije mogu djelovati kao “luksuz”, one su često razlika između manjeg incidenta i potpunog prekida rada. U praksi pouzdanosti, naglašava se i kultura učenja iz incidenata, gdje se nakon problema radi analiza i uvode mjere da se isti tip problema ne ponovi” [4].

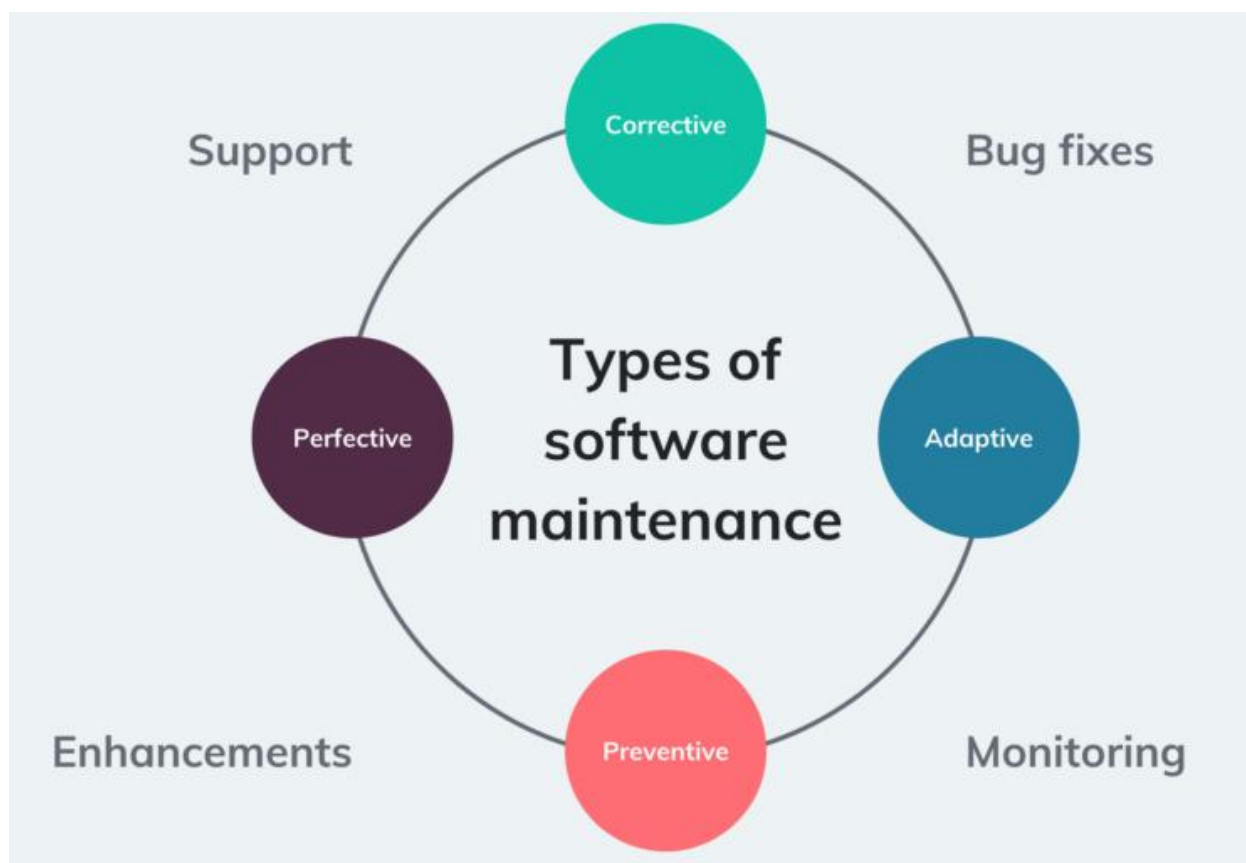
Deploy strategije su posebno važne kad sistem mora biti dostupan bez prekida. U takvim sistemima se teži raspoređivanju bez zastoja, gdje se stara verzija postepeno zamjenjuje novom. Međutim, i tu postoje rizici: ako nova verzija ima grešku, ona se može brzo proširiti. Zato su važni mehanizmi brzog povratka, a i dobra vidljivost kroz monitoring i logove.

“Dodatno, deploy je usko vezan za konfiguraciono upravljanje. Ako se ne zna tačno šta je raspoređeno, teško je održavati sistem. IEEE standard za konfiguraciono upravljanje govori o potrebi da se identifikuju konfiguracione stavke, kontrolišu promjene i vodi status, upravo da bi se u svakom trenutku moglo odgovoriti: koja verzija radi gdje i pod kojim uslovima [5]. Kad je to urađeno, održavanje postaje mnogo racionalnije, a manje “gašenje požara”.

3. ODRŽAVANJE I EVOLUCIJA SOFTVERA

3.1. Vrste održavanja i tipični okidači promjena

Održavanje se često dijeli na više vrsta. Najčešće se govori o korektivnom održavanju (ispravka grešaka), adaptivnom (prilagođavanje promjenama okruženja), perfekcionom (poboljšanja i optimizacije) i preventivnom (aktivnosti koje smanjuju rizik budućih problema). Važno je da ova podjela nije samo teorija, nego pomaže timu da razumije zašto se nešto radi i kako to planirati. “Standard za održavanje detaljnije opisuje održavanje kao proces, i naglašava da se definišu vrste održavanja i aktivnosti koje se sprovode u okviru održavanja” [5]. Tipični okidači promjena su vrlo “životni”. Na primjer, korisnici traže novu funkcionalnost, pa se radi perfekciono održavanje. Drugi primjer je promjena zakona, kao što je fiskalizacija ili zaštita podataka, pa sistem mora da se prilagodi, što je adaptivno održavanje. Treći primjer su bezbjednosne prijetnje i ranjivosti, gdje se radi korekcija i preventivne mjere, jer je cilj smanjiti šansu za zloupotrebu.



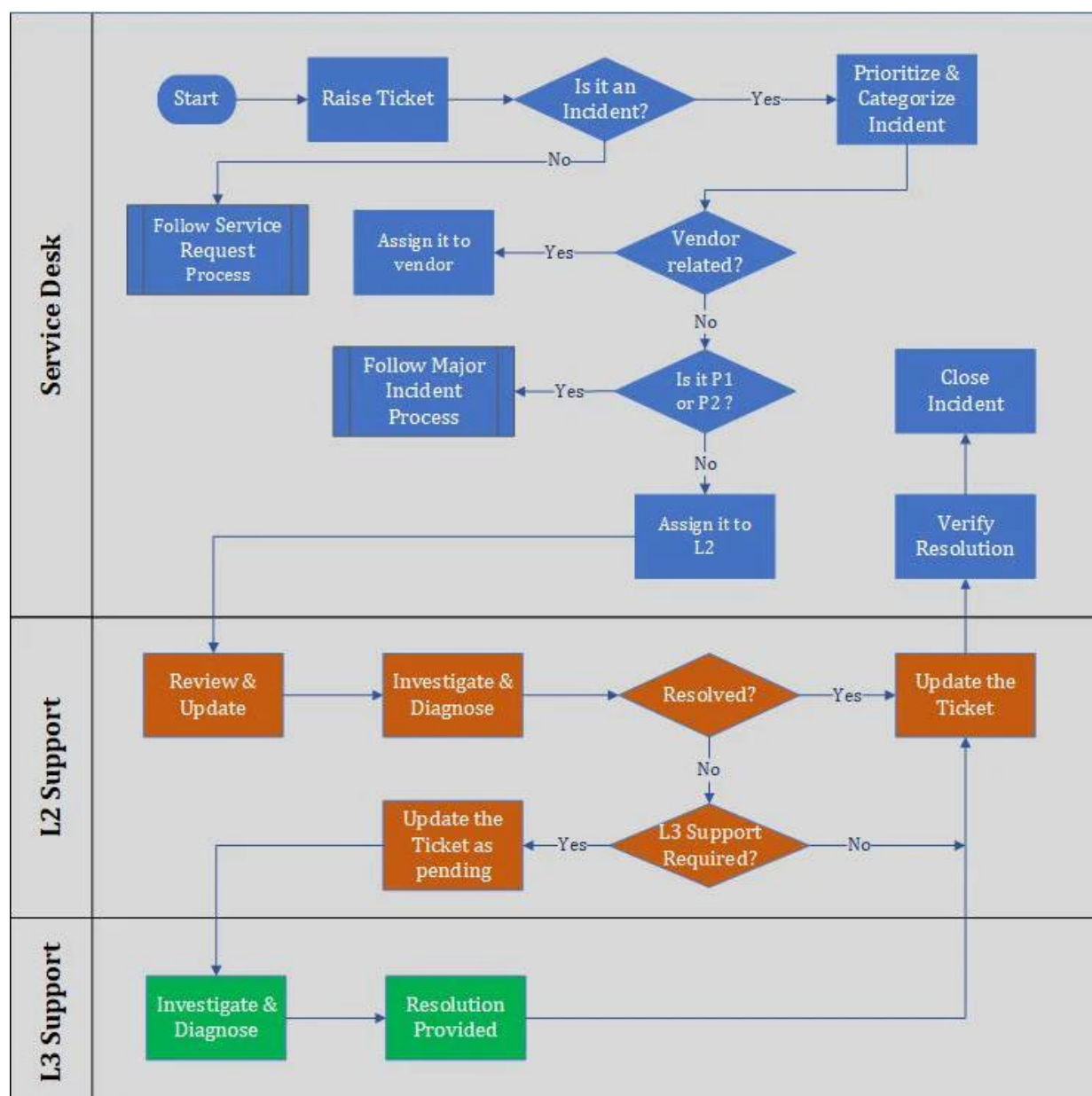
Slika 6 - Vrste održavanja softvera

“Jedna od važnih stvari u održavanju je da promjene dolaze u postojećem sistemu koji već radi i ima korisnike. Zbog toga održavanje nosi dodatni pritisak: ne smije se previše poremetiti ono što radi, a opet se mora mijenjati. U standardima životnog ciklusa održavanje je dio šireg okvira procesa, gdje se održavanje povezuje sa radom sistema i sa upravljanjem promjenama” [5].

U praksi to znači da tim mora stalno balansirati. Ako se radi samo “hitno popravljjanje”, sistem postaje haotičan. Ako se radi samo “sređivanje koda”, korisnici će osjećati da se ništa novo ne isporučuje. Najzreliji pristup je da se održavanje planira, da se dio kapaciteta tima uvijek ostavi za greške i incidente, a dio za unapređenja, uz jasne prioritete.

3.2. Upravljanje greškama, incidentima i povratnim informacijama korisnika

Greške su neizbježne, čak i u dobrim sistemima. Razlika je u tome kako se greške rješavaju i koliko sistem i tim mogu brzo da se oporave. U praksi se često razlikuju bug (greška u funkciji) i incident (prekid ili degradacija usluge u produkciji). Incidenti su obično hitniji jer utiču na korisnike odmah, dok bug može biti i manje kritičan.



Slika 7 - Dijagram toka upravljanja incidentima u IT sistemima

“U savremenim praksama pouzdanosti, mnogo se pažnje posvećuje načinu reagovanja na incidente, on-call režimu, definisanju odgovornosti i postmortem analizi, gdje se nakon incidenta traži uzrok i uči iz greške, bez traženja krivca” [4]. Poenta postmortema je da se uvedu promjene u sistem i proces, npr. bolji monitoring, bolji testovi, jasnija procedura deploy-a, kako se incident ne bi ponovio.

Povratne informacije korisnika su poseban kanal održavanja. Nekad korisnici prijave problem jasno, ali često prijave simptom: “ne radi mi plaćanje” ili “sporo učitava”, a tim mora otkriti stvarni uzrok. Tu se vidi koliko je važno da sistem ima dobre logove i metrike. Ako nema podataka, tim je primoran da pogađa, a to produžava vrijeme do rješenja.

“U IT servis menadžmentu, okviri kao ITIL daju terminologiju i opšte smjernice kako se organizuju incidenti, zahtjevi i promjene, sa ciljem da se usluga korisnicima održi stabilnom i predvidivom” [7]. ITIL ne rješava tehničke detalje, ali pomaže da se razumije da podrška nije improvizacija, nego proces. U praksi, i mali tim može primijeniti tu logiku kroz jasnu evidenciju tiketa, prioritizaciju i komunikaciju sa korisnicima.

3.3. Tehnički dug, refaktorisanje i modernizacija sistema

“Tehnički dug je pojam koji se često koristi da opiše situaciju kada se, zbog brzine ili nedostatka vremena, napravi rješenje koje radi, ali nije idealno. Dug se kasnije “plaća” kroz teže održavanje, više grešaka i sporije promjene. U knjigama o softverskom inženjeringu se naglašava da evolucija sistema vremenom postaje složenija i da su promjene teže ako se arhitektura i kod ne održavaju” [1]. To je jedna od najvažnijih lekcija za nekoga ko tek ulazi u ovu oblast: nije dovoljno da softver danas radi, bitno je da može da se mijenja sutra.

Refaktorisanje je jedan od načina da se tehnički dug smanji. To je poboljšanje strukture koda bez promjene spoljnog ponašanja. Na primjer, ako je neki dio koda predugačak i nepregledan, može se podijeliti na funkcije; ako su nazivi loši, mogu se popraviti; ako je logika duplirana, može se centralizovati. Iako refaktorisanje djeluje kao “nevidljiv posao”, ono se direktno isplati kroz lakše bug fixeve i stabilnija izdanja.

Modernizacija ili re-inženjering sistema postaje tema kada je sistem već star i teško ga je održavati. Nekad je problem u tome što tehnologija više nije podržana, nekad je problem u arhitekturi, a nekad u tome što su zahtjevi toliko promijenjeni da stari dizajn više ne odgovara. Tada se može raditi postepena modernizacija: zamjena dijelova sistema, migracija baze, prelazak na novi način deploy-a, ili uvođenje novih servisa.

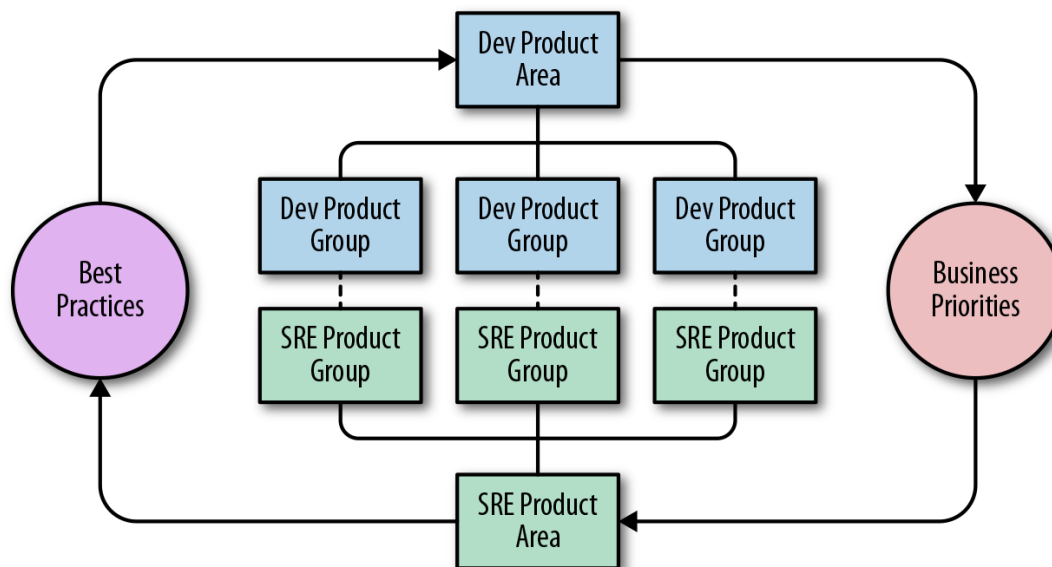
“Kvalitetni modeli kvaliteta pomažu i ovdje, jer se može jasno definisati šta želimo poboljšati. Na primjer, ako je cilj poboljšati održivost, možemo gledati koliko je sistem razumljiv, koliko je testiran, koliko brzo se može napraviti promjena i koliko često promjena pravi nove probleme. ISO model kvaliteta eksplicitno uključuje održivost kao osobinu proizvoda, što pomaže da se održavanje posmatra kao dio kvaliteta, a ne kao trošak” [7].

4. ORGANIZACIJA PODRŠKE I MJERENJE USPJEŠNOSTI

4.1. Operativna podrška i prakse iz SRE i ITIL okvira

“Operativna podrška obuhvata aktivnosti koje omogućavaju da sistem radi stabilno: monitoring, reakcija na incidente, upravljanje kapacitetom, backup i oporavak, kao i redovno održavanje okruženja. U standardu održavanja se naglašava da održavanje nije isto što i operacije, ali se u praksi te dvije oblasti stalno dodiruju, jer se problemi u radu često rješavaju kroz promjene u softveru” [5]. Zato mnoge organizacije pokušavaju da povežu razvoj i operacije kroz zajedničke ciljeve.

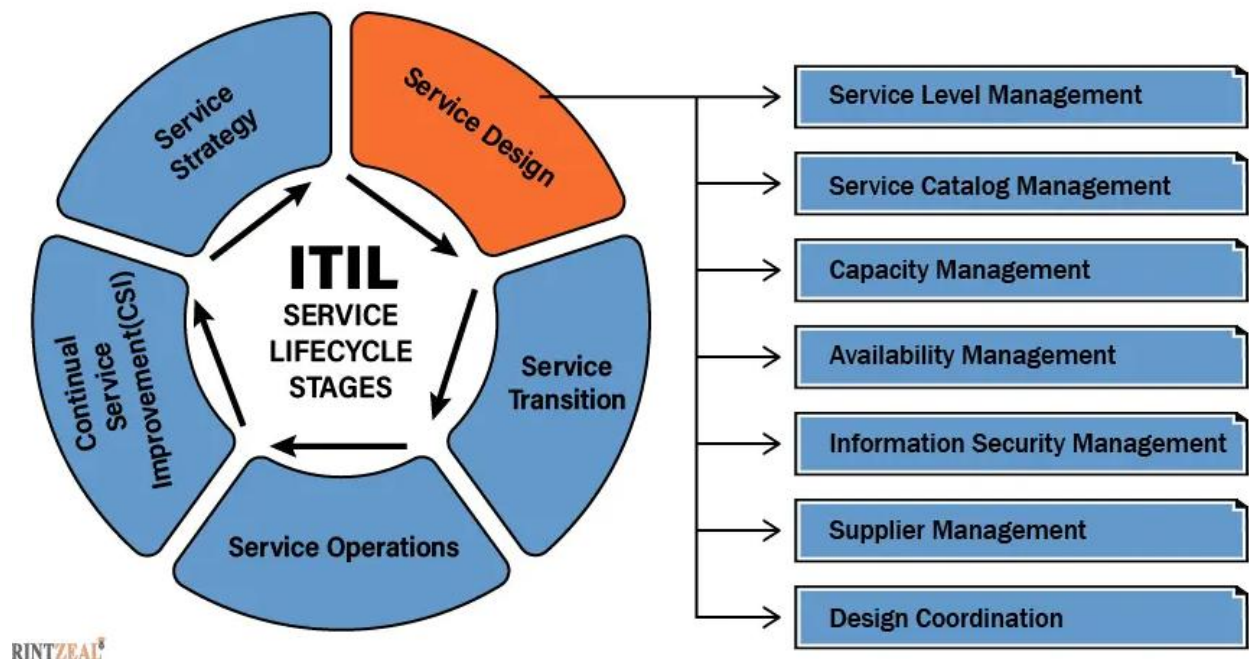
“SRE (Site Reliability Engineering) pristup, koji je popularizovan kroz iskustva Google-a, uvodi ideju da se pouzdanost tretira kao inženjerski problem, i da se sistem projektuje i održava tako da se može pratiti, skalirati i oporaviti” [4]. Ono što je meni kao studentu posebno razumljivo u ovom pristupu jeste naglasak na činjenici da sistem “živi” u produkciji i da je normalno da se nešto desi, ali je bitno koliko brzo i koliko mirno se reaguje.



Slika 8 - Dijagram SRE modela u Google pristupu pouzdanosti sistema

“ITIL, s druge strane, dolazi iz svijeta upravljanja IT uslugama i daje širi okvir, gdje je fokus na vrijednosti za korisnika, na procesima i na organizaciji rada” [7]. U realnim firmama se često kombinuje: SRE daje tehničke prakse i kulturu pouzdanosti, a ITIL daje jezik i strukturu za rad sa incidentima, promjenama i zahtjevima.

Dobra organizacija podrške ne mora biti ogromna. I mali tim može imati jasne uloge, npr. ko je dežurni, ko odobrava puštanje kritičnih promjena, ko komunicira sa korisnicima. Ključ je da se to dogovori unaprijed, jer u incidentu je najgore kad niko ne zna ko šta radi. Kad se proces postavi, održavanje postaje manje stresno, a sistem stabilniji.



Slika 9 - Lista ITIL procesa i faze životnog ciklusa ITIL usluga

4.2. Praćenje, logovanje i metrika isporuke i stabilnosti

Ako nešto ne mjerimo i ne pratimo, teško možemo reći da li se poboljšavamo. “U operativnom smislu, monitoring i logovanje su osnov da se uoči problem prije nego što ga korisnici masovno prijave. U SRE praksi se naglašava značaj monitoringa i jasnih signala koji govore da je usluga ugrožena” [4]. To ne znači da treba pratiti sve moguće metrike, nego da treba pratiti ono što stvarno govori o zdravlju sistema.

Pored operativnih metrika, u DevOps istraživanjima se često koriste metrike koje mjere performanse isporuke. U knjizi “Accelerate” se govori o tome da se performanse isporuke mogu mjeriti kroz metrike kao što su učestalost isporuka, vrijeme potrebno od promjene do produkcije, procenat promjena koje prave problem i vrijeme oporavka nakon incidenta [4]. Poenta ovih metrika nije da se “pritiska tim”, nego da se vidi gdje je usko grlo i gdje treba ulagati. Na primjer, ako je vrijeme od promjene do produkcije dugo, možda pipeline ima previše ručnih koraka. Ako je procenat neuspješnih promjena visok, možda testovi nisu dovoljno dobri ili se promjene uvode prevelikim paketima.

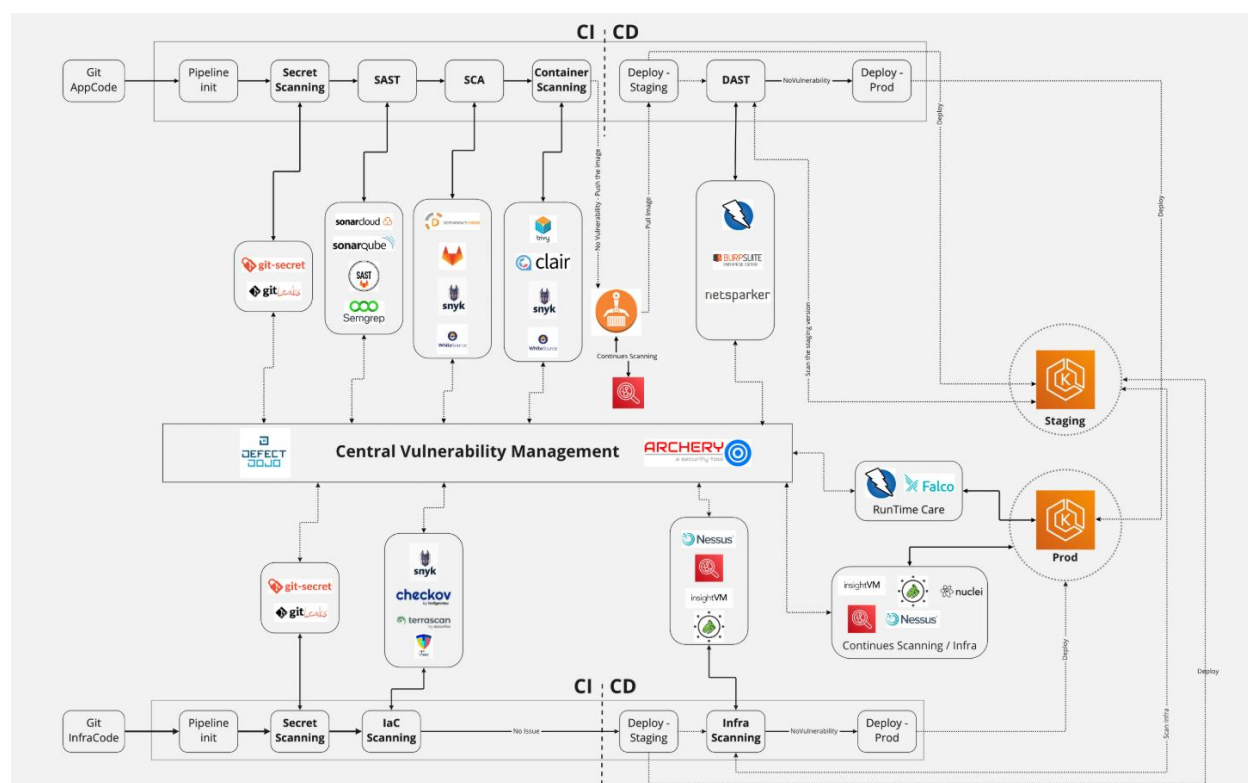
Metrike treba tumačiti pažljivo. Ako se gledaju izolovano, mogu da prevare. Na primjer, visoka učestalost deploy-a je dobra samo ako sistem ostaje stabilan. Zato se često naglašava balans između brzine i stabilnosti, a ne “trka” u jednom broju. U praksi, metrika treba da bude alat za razgovor i poboljšanje, a ne alat za kaznu.

Logovanje je još jedan važan dio. Dobar log nije samo “print”, nego strukturisan zapis koji pomaže da se rekonstruše šta se desilo. Kad se incident desi u 2 ujutro, logovi su često jedini trag. Zato se

kvalitet logova i praćenja može posmatrati kao dio kvaliteta proizvoda, jer direktno utiče na održavanje i brzinu rješenja problema.

4.3. Bezbjednost i usklađenost kao dio isporuke i održavanja

Bezbjednost nije nešto što se radi samo jednom. Ranjivosti se otkrivaju stalno, biblioteke dobijaju zakrpe, pojavljuju se novi napadi, i sistem mora da se održava i sa te strane. Zbog toga se bezbjednost sve više smatra dijelom procesa isporuke i održavanja, a ne “posebnim projektom”. “Kontrole i preporuke iz okvira kao što je NIST SP 800-53 naglašavaju širok skup bezbjednosnih i privatnosnih kontrola, uključujući i oblasti koje se tiču konfiguracije, upravljanja promjenama, logovanja i incident responsea” [9]. Čak i ako firma ne radi formalnu usklađenost, logika kontrola može pomoći da se sistem učini sigurnijim.

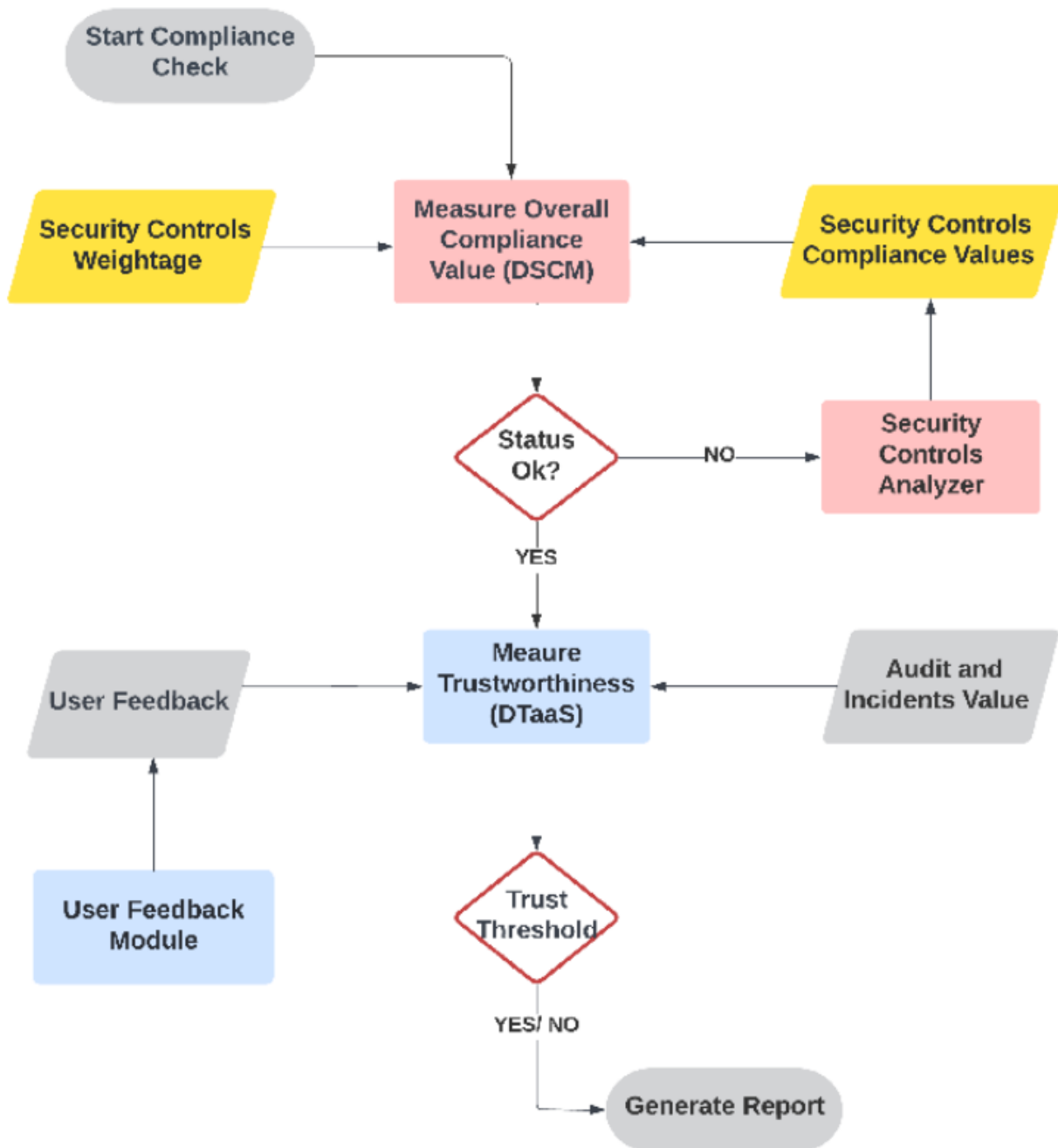


Slika 10 - OWASP DevSecOps smjernice (verzija 0.2)

U praksi, bezbjednosni zahtjevi često dolaze kao dio isporuke: treba uvesti jaču autentifikaciju, šifrovanje, ograničenja pristupa, ili audit logove. To znači da se moraju planirati i testirati kao i svaka druga promjena. Ako se bezbjednosna promjena isporuči nepažljivo, može izazvati pad sistema ili blokirati korisnike. “Zato su verifikacija i validacija bitne i u ovim slučajevima, jer promjena mora zadovoljiti i bezbjednosni cilj i funkcionalnu ispravnost” [11].

“Održavanje je posebno važno kod zavisnosti, kao što su biblioteke i frameworkovi. Ako tim dugo ne ažurira zavisnosti, može ostati na verzijama koje imaju poznate ranjivosti. Ali ažuriranje nosi rizik kompatibilnosti, pa je potrebno testiranje i kontrolisana isporuka. Tu se opet vidi veza između isporuke i održavanja: dobar CI/CD i dobar testni skup omogućavaju da se bezbjednosne zakrpe isporuče brže i sigurnije” [2].

“Usklađenost (compliance) može biti obaveza u određenim industrijama. Čak i kad nije formalna obaveza, mnoge organizacije imaju interne politike: kako se čuvaju podaci, ko ima pristup, kako se čuva audit trag. I tu isporuka i održavanje igraju ulogu, jer svaka promjena može uticati na usklađenost. Zbog toga je evidencija promjena, konfiguracije i izdanja više od administracije, ona je osnova da se sistemom upravlja odgovorno” [10].



Slika 11 - Dijagram toka za mjerenje usklađenosti i pouzdanosti sistema

ZAKLJUČAK

Isporuka i održavanje softvera nisu “poslovi nakon programiranja”, nego dio istog inženjerskog problema: kako napraviti sistem koji radi pouzdano, može se mijenjati i daje vrijednost korisniku tokom vremena. U ovom radu se vidi da isporuka obuhvata planiranje izdanja, kontrolu promjena, automatizaciju provjera i sigurno puštanje u rad, dok održavanje obuhvata mnogo više od popravke grešaka, uključujući adaptaciju, unapređenje i preventivne mjere.

Posebno je važno razumjeti da dobar proces isporuke direktno olakšava održavanje. Ako su izdanja mala i česta, ako postoji CI/CD, ako je konfiguracija pod kontrolom i ako se sve prati kroz monitoring i logove, tada se greške rješavaju brže, a incidenti su blaži. S druge strane, ako se isporuka radi ručno i neuredno, održavanje postaje stalno gašenje požara i sistem postepeno gubi stabilnost.

Standardi i okviri koji su pomenuti u radu daju zajednički jezik i osnovne principe: životni ciklus procesa i održavanje kao proces, kvalitet i modeli kvaliteta, konfiguraciono upravljanje, verifikacija i validacija, kao i organizacija IT usluge i pouzdanosti. Oni ne zamjenjuju iskustvo, ali pomažu da se praksa ne svodi na improvizaciju.

POPIS SLIKA

Slika 1 - 7 faza životnog ciklusa razvoja softvera (SDLC)	2
Slika 2 - Dijagram DevOps procesa isporuke softvera	3
Slika 3 - Upravljanje izdanjima sa dijagramom toka u pet koraka	4
Slika 4 - Dijagram CI/CD procesa isporuke softvera	5
Slika 5 - CI/CD proces sa GitHub Actions na Azure Web App	5
Slika 6 - Vrste održavanja softvera	7
Slika 7 - Dijagram toka upravljanja incidentima u IT sistemima	8
Slika 8 - Dijagram SRE modela u Google pristupu pouzdanosti sistema.....	10
Slika 9 - Lista ITIL procesa i faze životnog ciklusa ITIL usluga	11
Slika 10 - OWASP DevSecOps smjernice (verzija 0.2)	12
Slika 11 - Dijagram toka za mjerenje usklađenosti i pouzdanosti sistema	13

CITATNI IZVORI

- [1] R. S. Pressman i B. R. Maxim, Software Engineering: A Practitioner's Approach, New York, NY: McGraw-Hill Education, 2019.
- [2] J. Humble i D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Upper Saddle River, NJ: Addison-Wesley Professional, 2010.
- [3] I. Sommerville, Software Engineering, Boston, MA: Pearson, 2015.
- [4] B. Beyer, C. Jones, J. Petoff i N. R. Murphy, Site Reliability Engineering: How Google Runs Production Systems, Sebastopol, CA: O'Reilly Media, 2016.
- [5] IEEE, »IEEE Std 828-2012: IEEE Standard for Configuration Management in Systems and Software Engineering,« IEEE, New York, NY, 2012.
- [6] ISO/IEC/IEEE, »ISO/IEC/IEEE 14764:2022 Software engineering – Software life cycle processes – Maintenance,« International Organization for Standardization (ISO), Geneva, 2022.
- [7] ISO/IEC/IEEE, »ISO/IEC/IEEE 12207:2017 Systems and software engineering – Software life cycle processes,« International Organization for Standardization (ISO), Geneva, 2017.
- [8] AXELOS, ITIL Foundation: ITIL 4 Edition, London: TSO (The Stationery Office), 2019.
- [9] ISO/IEC, »ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models,« International Organization for Standardization (ISO), Geneva, 2011.
- [10] National Institute of Standards and Technology (NIST), »NIST SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations,« NIST, Gaithersburg, MD, 2020.
- [11] IEEE, »IEEE Std 1012-2016: IEEE Standard for System, Software, and Hardware Verification and Validation,« IEEE, New York, NY, 2017.